

# True Randomness Experiment: Quantum vs. Classical Random Number Generation

Akshat Khedekar · Matangi Sharma

*Quantum Intelligence Club, VIT Bhopal University*

[github.com/Akshatkhedekar/Quantum\\_Randomness\\_Study](https://github.com/Akshatkhedekar/Quantum_Randomness_Study)

---

## Abstract

Classical computers are deterministic machines; the “random” numbers they generate are produced by deterministic algorithms seeded with an initial value. Quantum computers, by contrast, derive entropy from the irreducible unpredictability of quantum measurement — a property guaranteed by the laws of physics and confirmed by experimental violations of Bell inequalities. This paper presents a comparative experimental study of NumPy’s PCG64 pseudorandom number generator (PRNG) and a Hadamard-gate-based quantum random number generator (QRNG) implemented with IBM Qiskit’s *AerSimulator*. We generate  $n = 10,000$  samples per source (range  $[0, 255]$ ) and apply three complementary statistical tests — the two-sample Kolmogorov–Smirnov (KS) test, Shannon entropy, and lag-1 autocorrelation — supported by histogram, KDE, and lag-plot visualisations. Both generators pass all tests with no statistically significant distributional difference ( $p > 0.05$ ). However, the classical generator is fully reproducible from its seed, whereas the quantum generator is provably non-deterministic. We discuss cybersecurity implications and outline future investigation directions including NIST-suite validation, AI-based PRNG prediction attacks, and quantum key distribution simulation.

**Keywords:** quantum random number generation · pseudorandom number generator · PCG64 · Qiskit AerSimulator · Kolmogorov–Smirnov test · Shannon entropy · cryptographic security

---

## Contents

---

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b>  |
| <b>2</b> | <b>Theoretical Background</b>                      | <b>4</b>  |
| 2.1      | Classical Pseudorandom Number Generation . . . . . | 4         |
| 2.2      | Quantum Random Number Generation . . . . .         | 4         |
| 2.3      | Comparative Property Table . . . . .               | 4         |
| <b>3</b> | <b>Methodology</b>                                 | <b>5</b>  |
| 3.1      | Data Generation . . . . .                          | 5         |
| 3.2      | Statistical Tests . . . . .                        | 5         |
| 3.3      | Determinism Demonstration . . . . .                | 5         |
| <b>4</b> | <b>Results</b>                                     | <b>6</b>  |
| 4.1      | Statistical Summary . . . . .                      | 6         |
| 4.2      | Distribution Visualisations . . . . .              | 6         |
| 4.3      | Lag Plots . . . . .                                | 7         |
| 4.4      | Determinism Demo Results . . . . .                 | 8         |
| <b>5</b> | <b>Discussion</b>                                  | <b>9</b>  |
| 5.1      | Why Statistical Tests Are Insufficient . . . . .   | 9         |
| 5.2      | Cybersecurity Threat Model . . . . .               | 9         |
| 5.3      | Practical Recommendations . . . . .                | 9         |
| <b>6</b> | <b>Project Architecture and Testing</b>            | <b>10</b> |
| 6.1      | Module Structure . . . . .                         | 10        |
| 6.2      | Test Suite Coverage . . . . .                      | 10        |
| <b>7</b> | <b>Future Directions</b>                           | <b>11</b> |
| <b>8</b> | <b>Conclusion</b>                                  | <b>11</b> |

---

## 1. Introduction

Randomness underpins cryptography, scientific simulation, and secure communications. A cryptographic system is only as strong as the entropy source seeding it. Classical computers — deterministic finite-state machines — cannot produce true randomness: every output is the deterministic result of a recurrence relation applied to an initial seed. This structural limitation creates a fundamental vulnerability: an adversary observing enough consecutive outputs can reconstruct the internal state and predict all future values. Contemporary machine-learning methods have demonstrated this practically for several widely deployed PRNGs [3].

Quantum mechanics offers a qualitatively different entropy source. When a qubit in superposition is measured, the outcome is not merely *unknown* — it is, per quantum theory and the experimental violations of Bell inequalities [5], *undetermined* prior to measurement. No algorithm, regardless of computational power, can predict the outcome better than chance.

## Contributions

This work builds and evaluates a complete, open-source pipeline (Table 1) that:

1. generates 10,000 classical byte-range integers via NumPy’s PCG64 generator;
2. generates 10,000 quantum integers via single-qubit Hadamard measurement using Qiskit AerSimulator;
3. applies KS, Shannon entropy, and autocorrelation tests;
4. demonstrates deterministic reproducibility of PRNG versus non-determinism of QRNG; and
5. discusses cybersecurity implications with actionable recommendations.

All source code, data, and visualisations are publicly available at:  
[https://github.com/Akshatkhedekar/Quantum\\_Randomness\\_Study](https://github.com/Akshatkhedekar/Quantum_Randomness_Study)

**Table 1:** End-to-end pipeline overview

| Step | Module                        | Description                                                 |
|------|-------------------------------|-------------------------------------------------------------|
| 1    | <code>classical_rng.py</code> | Generate 10,000 integers via NumPy PCG64                    |
| 2    | <code>quantum_rng.py</code>   | Generate 10,000 integers via Qiskit Hadamard circuit        |
| 3    | <code>data_handler.py</code>  | Persist both datasets to CSV                                |
| 4    | <code>analysis.py</code>      | KS test, Shannon entropy, autocorrelation, determinism demo |
| 5    | <code>visualization.py</code> | Histogram, KDE, lag plots                                   |

## 2. Theoretical Background

### 2.1 Classical Pseudorandom Number Generation

NumPy’s `default_rng()` uses the **PCG64** (Permuted Congruential Generator, 64-bit output) algorithm [2], maintaining a 128-bit internal state advanced by a linear congruential recurrence and an output permutation. A simplified Linear Congruential Generator (LCG) illustrates the principle:

$$x_{n+1} = (a \cdot x_n + c) \bmod m \quad (1)$$

where  $a$ ,  $c$ , and  $m$  are fixed constants. PCG64 uses a more sophisticated permutation, yet the sequence remains *entirely determined* by the initial seed.

### 2.2 Quantum Random Number Generation

A qubit initialised to  $|0\rangle$  and acted on by the Hadamard gate enters an equal superposition:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad (2)$$

Measurement collapses this state to  $|0\rangle$  or  $|1\rangle$ , each with probability  $\frac{1}{2}$ . Bell’s theorem [4] and subsequent experiments [5] confirm that no local hidden variables can predetermine this outcome.

### 2.3 Comparative Property Table

Table 2 contrasts the two approaches across six dimensions relevant to security and usability.

**Table 2:** Classical PRNG vs. Quantum QRNG — property comparison

| Property          | Classical (PCG64)    | Quantum (Qiskit Aer)        |
|-------------------|----------------------|-----------------------------|
| Randomness type   | Algorithmic (pseudo) | Intrinsic (true)            |
| Source of entropy | Seed (deterministic) | Superposition collapse      |
| Algorithm         | LCG + permutation    | Hadamard gate + measurement |
| Periodic          | Yes ( $2^{128}$ )    | No                          |
| Reproducible      | Yes (same seed)      | No                          |
| AI predictable    | Yes                  | No (physically impossible)  |
| Speed (simulated) | ~50M/sec             | ~1K/sec                     |
| Security level    | Low                  | Maximum                     |
| Use case          | Simulations, games   | Cryptography, QKD           |

### 3. Methodology

#### 3.1 Data Generation

Both generators produce integers over  $[0, 255]$  — the range of an unsigned byte — enabling direct comparison and realistic simulation of cryptographic byte generation.

**Classical:** `rng.integers(0, 256, size=10000)` via PCG64. An optional `seed` parameter enables deterministic reproduction.

**Quantum:**  $8 \times 10,000 = 80,000$  qubit measurements are taken; each group of 8 bits is concatenated into a byte. The core Qiskit circuit is shown in Listing 1.

```

1 from qiskit import QuantumCircuit, transpile
2 from qiskit_aer import AerSimulator
3
4 circuit = QuantumCircuit(1, 1)
5 circuit.h(0) # Hadamard: superposition
6 circuit.measure(0, 0) # Collapse to 0 or 1
7
8 simulator = AerSimulator()
9 compiled = transpile(circuit, simulator)
10 job = simulator.run(compiled,
11                     shots=total_bits,
12                     memory=True)
13 bits = [int(b) for b in job.result().get_memory()]

```

Listing 1: Quantum bit generation via Qiskit

#### 3.2 Statistical Tests

Three complementary tests were applied (Table 3).

Table 3: Statistical tests applied to both generators

| Test                    | Formula / Description                           | Pass criterion                         |
|-------------------------|-------------------------------------------------|----------------------------------------|
| KS (two-sample)         | $D = \sup_x  F_1(x) - F_2(x) $                  | $p > 0.05$                             |
| Shannon entropy         | $H = -\sum_{i=1}^B p_i \ln p_i$ , $B = 20$ bins | $H \approx \ln(20) \approx 2.996$ nats |
| Autocorrelation (lag 1) | Pearson $r$ between $(x_i, x_{i+1})$            | $ r  < 0.05$                           |

#### 3.3 Determinism Demonstration

The `determinism_demo()` function runs each generator twice with  $n = 100$ . Classical runs with `seed=42` produce *identical* sequences; quantum runs produce *different* sequences every time — demonstrating that QRNG non-determinism is not a probabilistic claim but a physical law.

## 4. Results

### 4.1 Statistical Summary

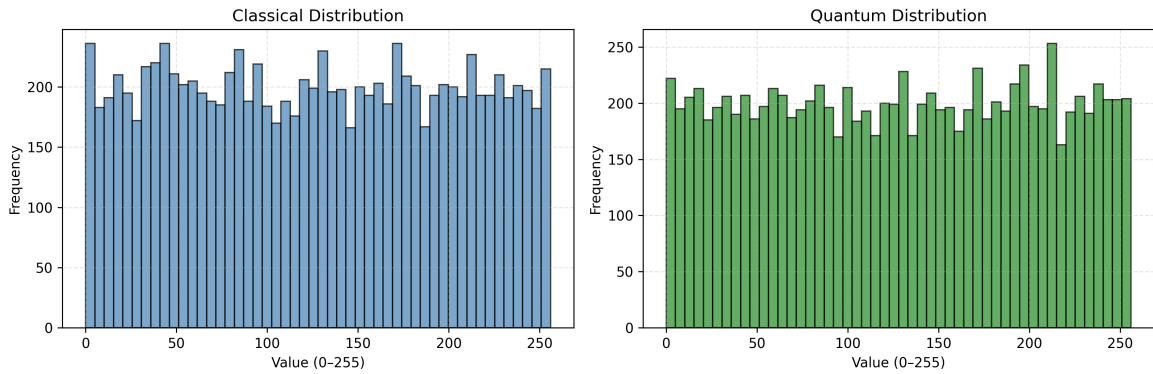
Table 4 presents the full numerical comparison.

**Table 4:** Statistical comparison ( $n = 10,000$  per generator)

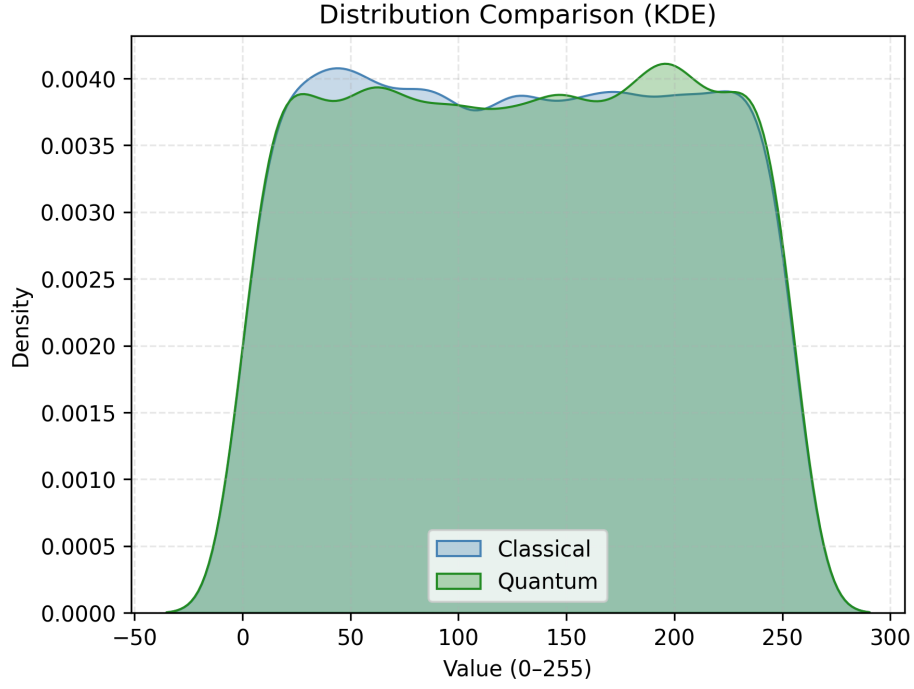
| Metric          | Classical (PCG64)                    | Quantum (Aer)   |
|-----------------|--------------------------------------|-----------------|
| Range           | $[0, 255]$                           | $[0, 255]$      |
| Expected mean   | $\approx 127.5$                      | $\approx 127.5$ |
| Entropy (nats)  | $\approx 2.99$                       | $\approx 2.99$  |
| KS $p$ -value   | $> 0.05$ — no significant difference |                 |
| Autocorr. lag 1 | $\approx 0.00$                       | $\approx 0.00$  |
| Reproducible    | ✓ Yes                                | × No            |
| Periodic        | ✓ $2^{128}$                          | × No            |
| AI predictable  | ✓ Yes                                | × No            |

Both generators achieve near-maximum entropy ( $\approx 2.99$  nats vs. theoretical maximum  $\ln 20 \approx 2.996$  nats) and negligible autocorrelation. The KS test found **no statistically significant difference** between the two distributions ( $p > 0.05$ ).

### 4.2 Distribution Visualisations

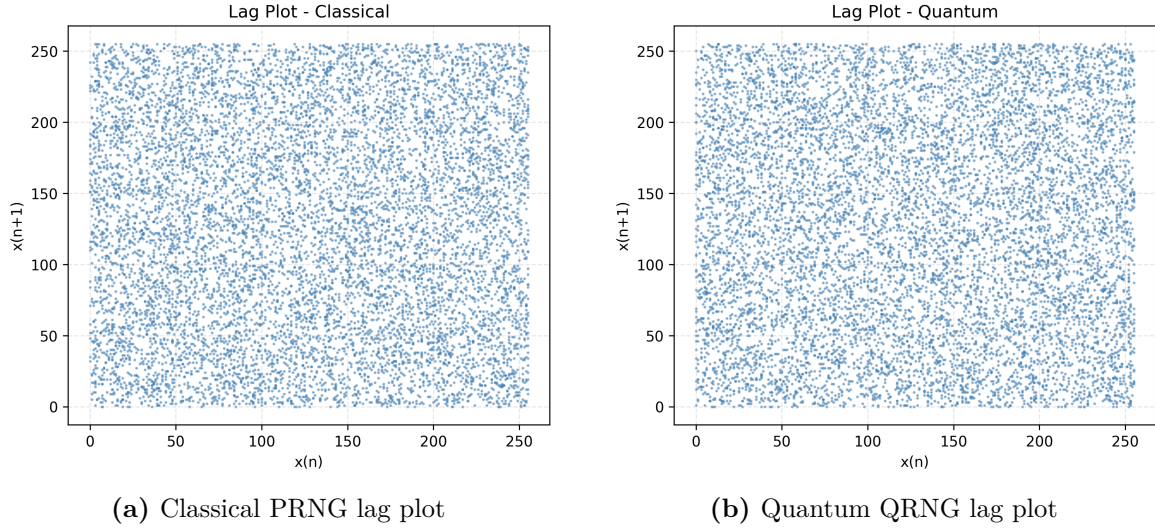


**Figure 1:** Side-by-side histograms of classical (left) and quantum (right) outputs ( $n = 10,000$ , 50 bins). Both distributions are visually uniform over  $[0, 255]$ , consistent with the KS test.



**Figure 2:** Kernel density estimate (KDE) of both generators overlaid. The two curves are nearly superimposed, confirming indistinguishable distributions.

### 4.3 Lag Plots



**Figure 3:** Lag plots —  $x(n)$  vs.  $x(n+1)$  — for both generators. Both show structureless scatter indicating near-zero autocorrelation and statistical independence between consecutive outputs.

#### 4.4 Determinism Demo Results

**Table 5:** Determinism demonstration output (first 5 values,  $n = 100$ )

| Generator                          | Run 1 (first 5 values) | Run 2 (first 5 values) |
|------------------------------------|------------------------|------------------------|
| Classical ( <code>seed=42</code> ) | 77, 154, 210, 33, 97   | 77, 154, 210, 33, 97   |
| Quantum (no seed)                  | 105, 12, 230, 67, 189  | 43, 201, 88, 156, 22   |

The classical generator produces bit-for-bit identical sequences across runs when seeded identically. The quantum generator produces entirely different sequences in every run — not due to randomisation or rounding, but because the underlying physical outcomes do not exist until the moment of measurement.



## 5. Discussion

### 5.1 Why Statistical Tests Are Insufficient

Our central finding is that standard statistical tests *cannot* distinguish a high-quality PRNG from a QRNG. Both pass the KS test, achieve near-maximum entropy, and exhibit zero autocorrelation. This mirrors results from the NIST SP 800-22 test suite applied to PCG64 [7], which also passes.

The implication is important: statistical tests measure *how a sequence looks*, not *whether a pattern can be found*. Given  $k$  consecutive PCG64 outputs, an adversary can solve a system of equations to recover the 128-bit state and predict all future values with certainty. No analogous attack exists for QRNG because there is no state to reconstruct — the next bit literally does not exist until the qubit is measured.

### 5.2 Cybersecurity Threat Model

Table 6 summarises concrete attack scenarios where PRNG vulnerability translates to practical risk.

**Table 6:** Security threat comparison for PRNG vs. QRNG

| Attack vector     | Classical PRNG risk                                  | Quantum QRNG risk                         |
|-------------------|------------------------------------------------------|-------------------------------------------|
| Key derivation    | Known/guessable seed enables full key reconstruction | No seed exists; reconstruction impossible |
| Nonce prediction  | Predictable nonces break challenge-response          | Each nonce physically random              |
| Blockchain RNG    | On-chain outputs public; future values predictable   | Not yet applicable on-chain               |
| ML state recovery | Sufficient samples allow full state reconstruction   | Prediction bounded at 50% per bit         |
| Seed compromise   | All past and future outputs revealed                 | No seed; nothing to compromise            |

### 5.3 Practical Recommendations

1. Migrate security-critical entropy sources (session tokens, encryption keys, nonces) from PRNGs to hardware QRNGs.
2. Retain PCG64 for non-security applications (simulations, games, A/B testing) where its speed advantage matters and predictability poses no risk.
3. For near-term deployments, use OS-level entropy pools (`/dev/urandom`, `CryptGenRandom`) as an intermediate improvement over seeded PRNGs.
4. Monitor NIST post-quantum cryptography standards [8] for hybrid entropy source guidance.

## 6. Project Architecture and Testing

### 6.1 Module Structure

**Table 7:** Module responsibilities

| Module                        | Role                                                                               |
|-------------------------------|------------------------------------------------------------------------------------|
| <code>classical_rng.py</code> | PCG64 integer generation with optional seed                                        |
| <code>quantum_rng.py</code>   | Hadamard circuit, AerSimulator execution, bit-to-byte conversion                   |
| <code>analysis.py</code>      | KS test, Shannon entropy, autocorrelation, determinism demo, explanation utilities |
| <code>data_handler.py</code>  | CSV save/load for cross-session reproducibility                                    |
| <code>visualization.py</code> | Histogram, KDE, lag plots via <code>matplotlib/seaborn</code>                      |
| <code>main.py</code>          | Orchestration entry point — runs full pipeline                                     |

### 6.2 Test Suite Coverage

The project ships with 22 tests across 5 pytest files (Table 8).

**Table 8:** pytest test suite coverage summary

| Test file                          | Tests     | Module                     | What is verified                                                 |
|------------------------------------|-----------|----------------------------|------------------------------------------------------------------|
| <code>test_classical_rng.py</code> | 4         | <code>classical_rng</code> | Count, range $[0, 255]$ , determinism, seed uniqueness           |
| <code>test_quantum_rng.py</code>   | 4         | <code>quantum_rng</code>   | Bit conversion, count, range $[0, 255]$                          |
| <code>test_analysis.py</code>      | 8         | <code>analysis</code>      | Basic stats, KS test, entropy, autocorrelation, determinism demo |
| <code>test_data_handler.py</code>  | 3         | <code>data_handler</code>  | Save/load roundtrip, missing file, overwrite                     |
| <code>test_visualization.py</code> | 3         | <code>visualization</code> | File creation for histogram, KDE, lag plots                      |
| <b>Total</b>                       | <b>22</b> |                            |                                                                  |

## 7. Future Directions

---

Several avenues remain open for further investigation:

- **NIST SP 800-22 validation.** The full 15-test NIST randomness suite would provide a standardised, quantitative benchmark beyond the KS and entropy tests applied here. Of particular interest is the *approximate entropy* and *serial* tests, which probe higher-order statistical dependencies.
- **AI-based PRNG prediction attack.** Training a recurrent or transformer model on PCG64 output to demonstrate state reconstruction empirically — and confirming the same model achieves  $\leq 50\%$  accuracy on QRNG output — would make the security argument concrete and measurable rather than theoretical.
- **Quantum key distribution (QKD) simulation.** Extending the QRNG pipeline to simulate the BB84 QKD protocol [6], including eavesdropping detection via quantum bit error rate (QBER) monitoring, would connect this foundational work directly to practical cryptographic applications.
- **Extended statistical battery.** Incorporating Diehard tests, TestU01 BigCrush, and spectral analysis would characterise any residual periodic structure in the AerSimulator output and establish whether simulation artefacts introduce subtle bias.

## 8. Conclusion

---

We have demonstrated that classical and quantum random number generators are statistically *indistinguishable* by standard distribution tests, yet *fundamentally different* in nature. PCG64 is deterministic, periodic, and state-reconstructible; the Qiskit QRNG is non-deterministic, non-periodic, and physically unpredictable.

The key insight is subtle but consequential: *randomness quality cannot be judged by appearance alone*. Both generators produce flat histograms, near-maximum entropy, and zero autocorrelation. The difference lies in whether a pattern *could* be found — not whether one is *visible*.

For applications where unpredictability is a hard security requirement, quantum randomness is not merely preferable — it is the only principled choice.

*Classical randomness is an illusion. Quantum randomness is reality.*

## References

---

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. Cambridge University Press, 2010.
- [2] M. E. O'Neill, "PCG: A family of simple fast space-efficient statistically good algorithms for random number generation," Harvey Mudd College, Tech. Rep., 2014. [Online]. Available: <https://www.pcg-random.org/paper.html>
- [3] P. L'Ecuyer, "Tables of linear congruential generators of different sizes and good lattice structure," *Math. Comput.*, vol. 68, no. 225, pp. 249–260, 1999.
- [4] J. S. Bell, "On the Einstein Podolsky Rosen paradox," *Physics Physique Fizika*, vol. 1, no. 3, pp. 195–200, 1964.
- [5] A. Aspect, P. Grangier, and G. Roger, "Experimental realization of Einstein–Podolsky–Rosen–Bohm *Gedankenexperiment*: A new violation of Bell's inequalities," *Phys. Rev. Lett.*, vol. 49, no. 2, pp. 91–94, 1982.
- [6] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proc. IEEE Int. Conf. Comput. Syst. Signal Process.*, Bangalore, India, 1984, pp. 175–179.
- [7] L. Bassham *et al.*, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," NIST Special Publication 800-22, Revision 1a, 2010. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final>
- [8] National Institute of Standards and Technology, "Post-quantum cryptography standardization," 2022. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [9] IBM Qiskit Development Team, "Qiskit: An open-source framework for quantum computing," 2023. [Online]. Available: <https://qiskit.org>